

# Auditable Keeps: Semantic Verification of Self-Improving Agent Decisions

Anonymous Authors

**Abstract**—A self-improving agent hands off decisions about which versions become the basis of later work. Matching file digests does not establish that recorded measurements justify those decisions. We present an executable decision-provenance profile linking candidate and parent versions, paired measurements, provisional resolutions, and final submission identity. Its offline verifier checks measurement reproducibility and declared decision rules, with coverage assessed separately from integrity. We evaluate its incremental checks using twelve author-designed packages derived from an executable fixture: nine faults, two valid controls, and one unsigned-reward rewrite control. Structural checking rejects one fault, structure plus file bindings rejects three, and complete verification rejects all nine; six faults therefore survive file-binding checks. Valid controls pass, as does the consistently rewritten reward, demonstrating the boundary between internal consistency and authenticity. Historical RSI-Exam rollout reports motivate the design but are not a controlled evaluation of auditing benefits. The result is a bounded, reproducible characterization of an artifact-level accountability mechanism for agent handoffs.

**Index Terms**—decision provenance, agent accountability, offline verification, research artifacts

## I. INTRODUCTION

An agent improving a program repeatedly proposes candidates, evaluates them, and decides what to retain. In RSI-Exam, an agent modifies an executable method and submits a final program for evaluation on unseen data [1]. A retained version becomes a parent of subsequent work. When another agent or organization receives the artifact, the final score alone cannot identify which measurements supported each decision or whether those measurements belong to the submitted version.

This is a concrete evidence problem. Our project’s historical preflight report records a task configuration in which visible evaluation results were written outside the exported methods directory and overwritten by later evaluations. The exported job preserved final hidden scores but not those intermediate measurements. The report also describes identical submitted Python sources whose bytecode caches differed from their snapshot caches. A naive whole-directory hash would identify them as different methods. These observations motivate explicit capture and identity rules; they are case reports, not estimates of prevalence.

We ask: *given an exported package, which decision inconsistencies can a recipient detect beyond checking structure and file bindings?* The answer requires relating bytes to a decision: a candidate and parent, oriented paired measurements, a reproducible statistic, and a resolution under a declared protocol. We contribute an executable profile for these relationships,

an offline verification procedure with explicit coverage and trust limits, and a controlled characterization of its additional checks.

The setting is relevant to decentralized agent accountability because a recipient can check a supplied package without querying the producing runtime. The experiments are local and single-agent; they do not evaluate network-scale governance, reputation robustness, or human audit speed.

## II. DECISION MODEL AND VERIFICATION

### A. Package and trust assumptions

A package contains a versioned record, snapshots, the submitted method, experiment and decision logs, and referenced results. Trajectory files may be bound by digest without embedding their contents. A producer constructs the record after a rollout; a recipient verifies it against the supplied directory.

We assume the recipient has the intended profile and verifier. The package producer and unsigned evaluation files are not authenticated by this mechanism. An adversary may change records, files, or both. Checks expose inconsistency relative to the received package, but cannot reconstruct a removed experiment when all its traces have been removed. They cannot establish that code was executed as claimed. A result binding associates a declaration with bytes; it is not a trusted execution receipt.

### B. The decision as an auditable unit

Represent a decision as

$$d = (p, c, E, \theta, v, a, r),$$

where  $p, c$  identify parent and candidate occurrences;  $E$  gives role-labeled result references and digests;  $\theta$  identifies the statistic, orientation, algorithm, seed, sample size, interval, and threshold;  $v$  is a statistical verdict;  $a$  is the disposition; and  $r$  identifies a resolved provisional decision when applicable.

Occurrences and contents have distinct identities: reverting may repeat earlier bytes. A method digest excludes bytecode caches according to the task’s staging convention, while a separate full-tree digest checks supplied directory bytes. Submitted identity is determined by the method-digest equivalence class and its canonical recorded representative.

For paired seed scores, oriented deltas are

$$\Delta_i = s_c(i) - s_p(i)$$

for a higher-is-better metric, with the sign reversed otherwise. The verifier recomputes the estimate and percentile-bootstrap

interval using the declared algorithm and random seed. Reproducibility does not establish nominal confidence coverage under adaptive reuse of visible seeds.

A verdict compares an interval to a threshold. A disposition applies the chosen protocol: a candidate can remain provisional pending confirmation. The record identifies which later event resolves that state. Protocol checks include unresolved provisional submission, contradictory dispositions, and insufficient confirmation sample counts.

### C. Recipient checks

For record  $R$  and supplied files  $F$ , acceptance with required complete coverage is

$$S(R) \wedge B(R, F) \wedge M(R, F) \wedge P(R) \wedge C(R, F).$$

$S$  checks structure,  $B$  checks file bindings,  $M$  checks cross-field identity and recomputed measurements,  $P$  checks decision rules, and  $C$  checks snapshot coverage. This describes the acceptance predicate; it is not a formal soundness proof.

Coverage is relative to the supplied snapshot directory. An extra unrecorded snapshot can leave integrity passing while coverage becomes partial; requiring complete coverage then rejects the package. A complete inventory can also coexist with an inconsistent statistic. Our evaluation preserves these different outcomes.

### D. Worked handoff

The fixture contains a parent with visible mean 4120 and a candidate with mean 3801.25. It records paired mean delta  $-318.75$  and interval  $[-382.5, -258.75]$ , supporting a revert for a higher-is-better score. Changing the estimate to  $-317.75$  in both the log and record projection, and updating the log hash, preserves structure and file bindings. The recipient nevertheless detects that the estimate cannot be recomputed, without a fresh policy execution.

## III. EVALUATION

### A. Design and reproducibility

We first replay ten existing development vectors, including one valid baseline, to inspect component boundaries. These were used to develop the implementation and provide no held-out performance estimate. Separately, we run twelve controlled packages derived from the committed gated fixture: nine faults, two valid controls, and one authenticity-boundary control.

The controlled manifest was committed before first execution, after inspection of the implementation and tests. This is an author-designed characterization, not a preregistered independent study. The verifier remains unchanged. Before each mutation, a clean copy must pass complete verification. Mutations operate on disposable copies; the report records changed-file hashes, source hashes, returned errors, and coverage.

We compare internal configurations on the same packages: **S**, the existing structural checker; **B**, structure plus referenced file and tree bindings; and **V**, full verification requiring complete coverage. **B** uses production digest functions but omits

TABLE I  
AUTHORED PACKAGES. A: ACCEPTS ITS CHECK SCOPE; R: REJECTS.

| Package change                                | S | B | V |
|-----------------------------------------------|---|---|---|
| None (valid control)                          | A | A | A |
| Excluded caches differ; full hashes refreshed | A | A | A |
| Reward and declaration rewritten together     | A | A | A |
| Unrecognized version status                   | R | R | R |
| Referenced measurement file removed           | A | R | R |
| Recorded orientation reversed                 | A | A | R |
| Recorded estimate increased by one            | A | A | R |
| Parent/candidate measurements exchanged       | A | A | R |
| Submitted provisional left unconfirmed        | A | A | R |
| Submission changed; version claim retained    | A | A | R |
| Recorded snapshot removed                     | A | R | R |
| Unrecorded snapshot added                     | A | A | R |

cross-field identity, measurement recomputation, protocol, and coverage checks. These are internal ablations, not third-party product comparisons. **B** checks source logs, reward files, decision evidence, recorded snapshots, and the final tree. Limited-check acceptance means only those checks passed.

### B. Controlled outcomes

Table I reports every case. Decision changes are mirrored in the log and capsule, and the log hash is refreshed. For swapped measurements, role labels remain while referenced files and digests exchange places. For a changed submission, its hashes are refreshed while its claimed version remains unchanged. These semantic cases do not rely on stale hashes.

**S** rejects one of nine faults, **B** rejects three, and **V** rejects nine. Six faults pass **B** but fail **V**. Both valid controls pass all configurations. These counts characterize authored cases; they do not estimate detection rates on naturally occurring errors.

Reason-level results qualify this conclusion. Orientation and swapped measurements fail statistic reproduction; the changed estimate fails estimate reproduction; provisional submission triggers its protocol error; changed submission identity triggers a cross-field mismatch. The extra snapshot produces partial coverage with no integrity error. A missing snapshot is rejected, but the manifest predicted the wrong diagnostic prefix: `file:version:v2`, versus the actual `file:artifact:v2:missing_file`. All nine faults are refused, but only eight match their prespecified diagnostic target. Unexpected runtime exceptions abort execution rather than count as detections.

### C. Boundary and development findings

The unsigned reward control changes both the reward file and capsule reward to 0.5 and updates the digest. **V** accepts it. The verifier checks agreement with the supplied reward file; it neither authenticates the grader nor recomputes the reward from score details. This acceptance limits downstream trust claims.

Development replay shows why refusal alone is insufficient. The existing descendant-of-open-provisional vector is rejected by the full construction path because its referenced version

is absent, before the intended ancestry check is reached. The converter reports the intended provisional-ancestry violation. We do not count the end-to-end refusal as proof of the named verifier capability. The controlled study instead tests an existing submitted provisional version with confirmation removed, reaching its explicit protocol check.

#### D. Historical deployment context

Committed cohort tables report 18 verified records among 20 started instrument/helper trials, nine per arm. The two refusals concern submissions without corresponding snapshots. This supports historical feasibility of capture and explicit refusal, not a controlled improvement in auditability. Original job directories are not all packaged here, so the tables are not newly reproduced end to end.

The reward comparison has three instrument-favoring and seven helper-favoring blocks. Small sample size and incomplete source precision in later Opus blocks do not support a general efficacy conclusion. We retain this as deployment context: enforcing a decision rule has different consequences from recording and checking one.

### IV. RELATED WORK AND IMPLICATIONS

PROV provides entity, activity, and derivation semantics; PROV-CONSTRAINTS already defines consistency conditions over provenance histories [2]. Our task-specific profile adds measurement recomputation, candidate/parent roles, submission identity, and keep/confirmation rules. We do not claim to invent provenance validation.

in-toto provides cryptographic verification of software supply chains [3]. Our unsigned packages have a weaker authenticity boundary. A possible composition would attest a package while separately checking its decision semantics; we have not implemented or evaluated this. MLflow tracks experiment parameters, metrics, and artifacts [4]. Such storage can retain our inputs; the experiment concerns additional recipient checks, not measured superiority over tracking systems.

TRACE represents decisions and typed confidence information. A historical fixture run validates its conversion and import into ProofPress, which projects selected confidence fields as evidence. Import performs neither complete verification nor human approval. A deployment must explicitly arrange verification and authorization; schema compatibility alone does not establish governed reuse.

A networked consumer could condition acceptance on verification outcomes before adding an artifact to its lineage. Authentication, authorization, independent witnessing, and resistance to collusion remain separate requirements. The Agent-Native Research Artifact companion [5] links claims to manifests, code, results, and the abandoned reward-centered framing, supporting inspection of the research argument.

### V. LIMITATIONS AND CONCLUSION

This is an internal characterization on a synthetic fixture, with author-selected faults and shared checking components. It does not demonstrate generalization to unseen tasks, lower

audit effort, independent implementer agreement, or attack resistance. Historical rollouts supply motivation and feasibility at weaker reproducibility strength. Coverage cannot reveal wholesale removal of an experiment, and internally consistent rewriting can pass. The statistical rule is checked as declared; its scientific adequacy is not certified.

Six authored faults survive structure and file binding but fail complete verification, while a consistently rewritten unsigned reward passes. These results delimit our profile's contribution: recipients can check specified decision relationships in supplied evidence, while authenticity and experimental truth require additional mechanisms.

### REFERENCES

- [1] Aiming Lab, *RSI-Exam*, 2026. [Online]. Available: <https://github.com/aiming-lab/RSI-Exam>
- [2] J. Cheney, P. Missier, and L. Moreau, Eds., *Constraints of the PROV Data Model*, W3C Recommendation, Apr. 2013.
- [3] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, *in-toto: Providing farm-to-table guarantees for bits and bytes*, in *USENIX Security*, 2019, pp. 1393–1410.
- [4] MLflow, *ML Experiment Tracking*. [Online]. Available: <https://mlflow.org/docs/latest/ml/tracking/>
- [5] ARA Labs, *Agent-Native Research Artifact*. [Online]. Available: <https://github.com/ARA-Labs/Agent-Native-Research-Artifact>